

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase 9: Verificación de Software

Verificación de Software

¿Cómo saber si nuestro código se comporta como debería?



Verificación de Software

Los **errores de software** pueden costar vidas humanas y grandes pérdidas económicas.



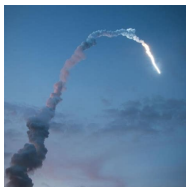
Therac-25, 3 muertos (1985)



Airbus A300, 264 muertos (1994)



Knight Capital, 440M USD (2012)



Ariane 5 (1996)



Y2K (1999/2000)



MS Zune (2008/2009)

<http://www.cse.psu.edu/~gxt29/bug/softwarebug.html>

Verificación de Software

- ▶ **Pruebas dinámicas:** requieren ejecutar el programa para detectar fallos en su comportamiento.
- ▶ **Verificación estática:** analizan el código, la documentación, etc. sin ejecutar el software.
- ▶ **Métodos formales:** uso de métodos matemáticos rigurosos para demostrar que un sistema cumple con sus especificaciones.

Pruebas dinámicas	Verificación estática	Métodos formales
Unidad	Revisión de código	Verificación formal
Integración	Análisis estático	Model checking
Sistema	Inspecciones	
Aceptación		
Regresión		
Desempeño		
Seguridad		

Verificación de Software: Testing

El testing puede demostrar la presencia de errores; nunca su ausencia

La forma más difundida y sencilla es el **testing de unidad**:

- ▶ Probar cada módulo de forma independiente (por ejemplo, una función o una clase).
- ▶ Definir pares de entrada y salida esperada para cada módulo.
- ▶ Ejecutar el módulo con cada entrada y verificar que la salida coincida con lo esperado.

Siempre procuramos que las entradas seleccionadas representen de manera fiel al conjunto total de entradas.

Idealmente, los tests debe diseñarlos alguien que no haya programado el software.

Verificación de Software: Testing

Ejemplo: testing de unidad para la función raíz cuadrada entera.

Entrada	Salida esperada
1	1
2	1
4	2
9	3
15000	122
15129	123
...	...

Verificación de programas

Ejercicio. (1) Pensar y escribir (en papel) pares de entrada/salida para realizar tests de unidad a las siguientes funciones:

- (a) `suma(xs)`, que dada una lista de enteros `xs` devuelve la suma de todos sus elementos.
- (b) `es_primo(n)`, que devuelve `True` si `n` es primo y `False` en caso contrario.

Tests de unidad en Python

Python tiene una [biblioteca para realizar tests de unidad](#) que nos permite comparar el resultado de evaluar una expresión (por ej., el resultado de llamar a una función) contra un valor.

```
# Importamos la biblioteca de tests
import unittest
from suma_lista import suma    # Importamos el código a testear.

# Creamos una clase en la que incluiremos los tests.
class TestSumaLista(unittest.TestCase):
    # TestSumaLista es subclase de unittest.TestCase.

    # Cada test de unidad debe comenzar con la palabra test.
    def test_todos_positivos(self):
        self.assertEqual(suma([1,1,1,1]), 4)
        self.assertEqual(suma([1,2,3,4]), 10)

    def test_todos_negativos(self):
        self.assertEqual(suma([-1,-1,-1,-1]), -4)
        self.assertEqual(suma([-1,-2,-3,-4]), -10)

    def test_positivos_negativos(self):
        self.assertEqual(suma([1,2,3,-3]), 3)
        self.assertEqual(suma([1,-1,1,-1]), 0)

    [...]

unittest.main()
```

Tests de unidad en Python

Otro ejemplo: tests para la función `es_primo(n)`.

```
import unittest
from numeros_primos import es_primo

class TestPrimos(unittest.TestCase):

    def test_primos(self):
        self.assertTrue(es_primo(5))
        self.assertTrue(es_primo(37))
        self.assertTrue(es_primo(101))

    def test_no_primos(self):
        self.assertFalse(es_primo(8))
        self.assertFalse(es_primo(44))
        self.assertFalse(es_primo(100))

    [...]

unittest.main()
```

<https://docs.python.org/3/library/unittest.html>

Tests de unidad en Python

Ejercicio. (2) Descargar los archivos `suma_lista.py` y `numeros_primos.py`, y escribir tests de unidad en Python para las funciones `suma_lista(xs)` y `es_primo(n)` usando los valores de entrada/salida elegidos en el ejercicio 1.

¡No mirar cómo están implementadas las funciones!

```
import unittest
from suma_lista import suma

class TestSumaLista(unittest.TestCase):
    [...]
```

```
import unittest
from numeros_primos import es_primo

class TestPrimos(unittest.TestCase):
    [...]
```

Depuración de programas

La **depuración** (o *debugging*) es el proceso de encontrar y corregir errores de programación.

```
import unittest
from numeros_primos import es_primo

class TestPrimos(unittest.TestCase):

    def test_uno(self):
        self.assertFalse(es_primo(1))

    [...]
```

```
=====
FAIL: test_uno (__main__.TestPrimos)
-----
Traceback (most recent call last):
  File "clase6/src/numeros_primos_test.py", line 7, in test_uno
    self.assertFalse(es_primo(1))
AssertionError: True is not false
-----

Ran 1 test in 0.000s

FAILED (failures=1)
```

Depuración de programas

Primer paso: localizar el error

```
def es_primo(n):  
    for i in range(2,n): # { }  
        if n % i == 0:  
            return False  
    return True
```

Segundo paso: corregirlo

```
def es_primo(n):  
    if n == 1:  
        return False  
    else:  
        for i in range(2,n): # { }  
            if n % i == 0:  
                return False  
    return True
```

Depuración de programas

Ejemplo: queremos testear la biblioteca repeticiones, que exporta las siguientes funciones:

- ▶ `cant_apariciones(elem,xs)`: devuelve la cantidad de veces que elem aparece en xs.
- ▶ `dict_cantidad_apariciones(xs)`: devuelve un diccionario que tienen como claves los elementos de xs y como valores asociados la cantidad de veces que aparece cada uno.
- ▶ `mas_repetido(xs)`: Devuelve el elemento más repetido de xs (no vacía). Si hay más de uno con cantidad de apariciones máxima, devuelve el que aparece primero.

```
def test_mas_repetido(self):  
    lista = [1,2,3,4,1,4,2,4,1]  
    self.assertEqual(mas_repetido(lista),1)
```

```
=====
FAIL: test_mas_repetido (__main__.TestMasRepetido)
-----

Traceback (most recent call last):
File "test_mas_repetido.py", line 14, in test_mas_repetido
self.assertEqual(mas_repetido(lista),1)
AssertionError: 4 != 1

-----

Ran 1 test in 0.000s

FAILED (failures=1)
```

Depuración de programas

Ejemplo: queremos testear la biblioteca repeticiones, que exporta las siguientes funciones:

- ▶ `cant_apariciones(elem, xs)`: devuelve la cantidad de veces que elem aparece en xs.
- ▶ `dict_cantidad_apariciones(xs)`: devuelve un diccionario que tienen como claves los elementos de xs y como valores asociados la cantidad de veces que aparece cada uno.
- ▶ `mas_repetido(xs)`: Devuelve el elemento más repetido de xs (no vacía). Si hay más de uno con cantidad de apariciones máxima, devuelve el que aparece primero.

```
def test_mas_repetido(self):  
    lista = [1,2,3,4,1,4,2,4,1]  
    self.assertEqual(mas_repetido(lista),1)
```

```
def mas_repetido(xs):  
    elem_mas_repetido = xs[0]  
    cant_repeticiones_maxima = 1  
    cant_apariciones = dict_cantidad_apariciones(xs)  
    for elem in cant_apariciones.keys():  
        if cant_apariciones[elem] > cant_repeticiones_maxima:  
            elem_mas_repetido = elem  
            cant_repeticiones_maxima = cant_apariciones[elem]  
    return elem_mas_repetido
```

La función `mas_repetido(...)` se encuentra correctamente implementada.

Depuración de programas

Ejemplo: queremos testear la biblioteca repeticiones, que exporta las siguientes funciones:

- ▶ `cant_apariciones(elem,xs)`: devuelve la cantidad de veces que elem aparece en xs.
- ▶ `dict_cantidad_apariciones(xs)`: devuelve un diccionario que tienen como claves los elementos de xs y como valores asociados la cantidad de veces que aparece cada uno.
- ▶ `mas_repetido(xs)`: Devuelve el elemento más repetido de xs (no vacía). Si hay más de uno con cantidad de apariciones máxima, devuelve el que aparece primero.

```
def test_mas_repetido(self):  
    lista = [1,2,3,4,1,4,2,4,1]  
    self.assertEqual(mas_repetido(lista),1)
```

```
def cant_apariciones(elem, xs):  
    res = 0  
    i = 0  
    while i < len(xs) - 1:  
        if xs[i] == elem:  
            res = res+1  
        i = i+1  
    return res
```

La error está en `cant_apariciones(...)`.

Depuración de programas

Ejemplo: queremos testear la biblioteca repeticiones, que exporta las siguientes funciones:

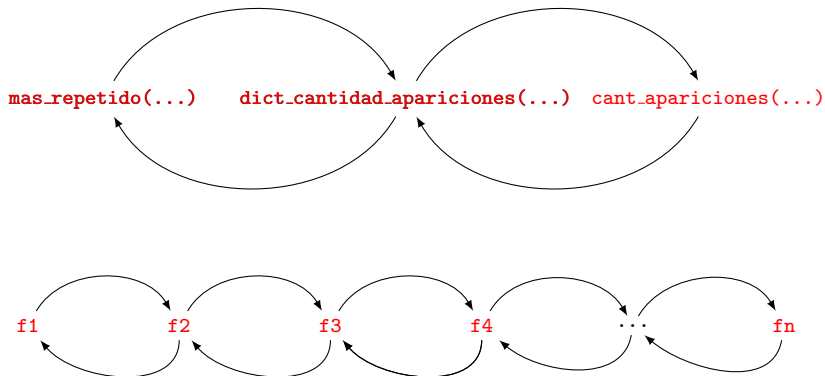
- ▶ `cant_apariciones(elem, xs)`: devuelve la cantidad de veces que `elem` aparece en `xs`.
- ▶ `dict_cantidad_apariciones(xs)`: devuelve un diccionario que tienen como claves los elementos de `xs` y como valores asociados la cantidad de veces que aparece cada uno.
- ▶ `mas_repetido(xs)`: Devuelve el elemento más repetido de `xs` (no vacía). Si hay más de uno con cantidad de apariciones máxima, devuelve el que aparece primero.

```
def test_mas_repetido(self):  
    lista = [1,2,3,4,1,4,2,4,1]  
    self.assertEqual(mas_repetido(lista), 1)
```

1. El programador introduce un **error** al escribir el código (ej: condición en un `while`).
2. Cuando el defecto se ejecuta, se genera una **infección** que corrompe el estado del programa (ej: cuando se ejecuta `cant_apariciones`).
3. La infección se propaga y se manifiesta como una **falla** (ej: cuando se ejecuta `mas_repetido`).

Depuración de programas

Al trabajar con programas grandes es difícil localizar errores. La **falla** puede producirse mucho después de la **infección**.



Depuración de programas

Una técnica para depurar programas consiste en [inspeccionar el estado del programa](#) (i.e., el valor de las variables) para recabar información que pueda guiarnos hacia el error.

Algunas herramientas, como *Spyder*, cuentan con un [debugger](#), que permite ejecutar un programa paso a paso e ir observando cómo va cambiando su estado.

```
def cant_apariciones(elem, xs):  
    ⇒ res = 0  
    i = 0  
    while i < len(xs) - 1:  
        if xs[i] == elem:  
            res = res + 1  
        i = i + 1  
    return res
```

Depuración de programas

Ejercicio: (3) Descargar el archivo `ej_depuracion.py`, escribir tests de unidad en Python para las funciones listadas a continuación y, en caso de producirse fallas, depurarlas. (Todas contienen al menos un error.)

Importante: escribir y correr los tests antes de mirar las funciones implementadas en `ej_depuracion.py`.

Interfaz de las funciones:

- ▶ `esta(elem, xs)`: devuelve True si `elem` está en `xs`; False en caso contrario.
- ▶ `repetido(elem, xs)`: devuelve True si `elem` aparece más de una vez en `xs`; False en caso contrario.
- ▶ `primer_repetido(xs)`: devuelve el primer elemento que aparece al menos dos veces en `xs`.

Precondición: hay al menos un elemento repetido en `lista`

```
import unittest
from ej_depuracion import esta, repetido, primer_repetido

class TestPertenenciaYRepeticiones(unittest.TestCase):

    def test_no_pertenencia(self):
        lst = [1,2,3,4]
        self.assertFalse(esta(5,lst))

    [...]
```

Repaso de la clase de hoy

- ▶ Verificación de programas. Tres enfoques:
 - ▶ Verificación formal
 - ▶ Revisión del código por pares
 - ▶ Testing
- ▶ No podemos estar 100 % seguros de que un programa es correcto.
- ▶ Técnica simple de debugging: seguimiento de variables durante ejecución.

Bibliografía complementaria:

- ▶ IPP, Apéndice A

Con lo visto, ya pueden resolver la sección 9 de la guía de ejercicios.